

Exam project

Fetal state classification

Student: Claudio Scamporlino

Student ID: [redacted for publication]

Course: Machine Learning and Deep Learning (UTSR)

Instructor: Prof. Noemi Scarpato

Project: Fetal state classification

Dataset: Cardiotocography - UCI ML Repository (ID: 193)

1. Introduction

This project is a classification experiment in Python. I worked with a medical dataset containing measurements extracted from fetal cardiocotocograms—the traces recorded during pregnancy to monitor fetal heart rate and uterine contractions.

I chose this dataset for several reasons. First, it has 2,126 samples and 21 numerical features—a size that suits both classical algorithms and deep learning. Having 3 classes (normal, suspect, pathologic) makes it more interesting than a simple binary problem. Finally, in a medical context, explainability is especially important: it helps identify which variables matter most in the model's decisions.

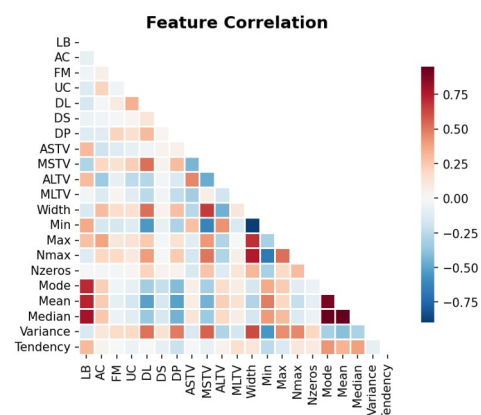
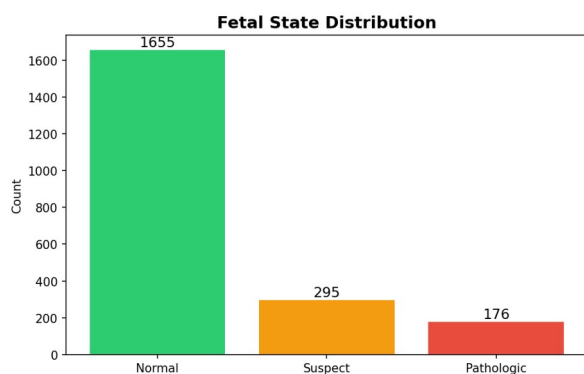
Beyond PCA, the three classifiers, evaluation metrics, and visualizations, I also included both a deep learning approach and SHAP-based explainability analysis.

2. The dataset

The dataset contains 21 features automatically measured from CTG traces. These include: LB (fetal heart rate baseline), AC (accelerations per second), FM (fetal movements), UC (uterine contractions), several types of decelerations (DL, DS, DP), short- and long-term variability measures (ASTV, MSTV, ALTV, MLTV), and fetal heart rate histogram statistics (Width, Min, Max, Mode, Mean, Median, Variance, etc.).

The target variable NSP classifies fetal state into three classes: Normal (1,655 samples, 77.8%), Suspect (295, 13.9%), and Pathologic (176, 8.3%). As we can see, the dataset is imbalanced: the vast majority of cases are normal, which reflects clinical reality but makes classifying minority classes more challenging.

One thing I noticed immediately from the correlation matrix is that many features are strongly correlated: Mode, Mean, and Median, for example, have correlations above 0.90. This suggests that PCA will be very effective at reducing dimensionality.



3. Preprocessing

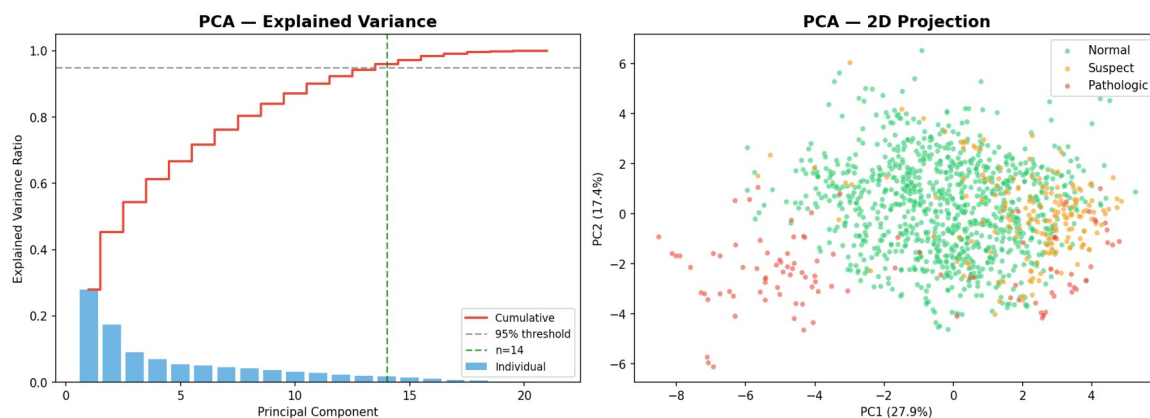
I split the data into three sets: training (70%), validation (20%), and test (10%), using stratification to preserve class proportions in each set. I used the same split for all experiments (shallow, deep learning, and SHAP) so the results are directly comparable.

I applied StandardScaler, which normalizes each feature by subtracting the mean and dividing by the standard deviation. This step is essential for PCA (which would otherwise be dominated by features with larger values) and for scale-sensitive algorithms such as SVM and Logistic Regression. I fit the scaler on the training set and then transformed validation and test with the same parameters.

4. Task 1 - PCA analysis

I applied PCA to the standardized training features. Analyzing the variance, I found that 14 components are enough to retain 95% of the original information. This reduces the problem size by removing redundancy due to strong correlations among the 21 original features.

I also created a 2-component projection to visualize the data. The scatter plot shows that the Pathologic class tends to form a partially separate cluster, while Normal and Suspect overlap extensively. This immediately suggested that separating Normal from Pathologic would be easier than separating Normal from Suspect.



I then used the 14 PCA components as input for all subsequent classifiers.

5. Task 2 - Classification algorithms

I implemented three classifiers, chosen because they represent different approaches to the problem:

Logistic Regression: I used it as a linear baseline. Despite its simplicity, it performs reasonably well because PCA has already “straightened” the data along the directions of maximum variance.

SVM with RBF kernel: I chose the RBF kernel because it introduces non-linearity and allows the model to draw curved decision boundaries. I set probability=True to compute probabilities and evaluate the model via ROC AUC, which is essential in clinical settings to assess system reliability.

Random Forest: An ensemble of 200 decision trees with maximum depth 10. I chose it because it is robust, handles imbalanced datasets well, and works well with SHAP (TreeExplainer).

6. Task 3 - Evaluation metrics

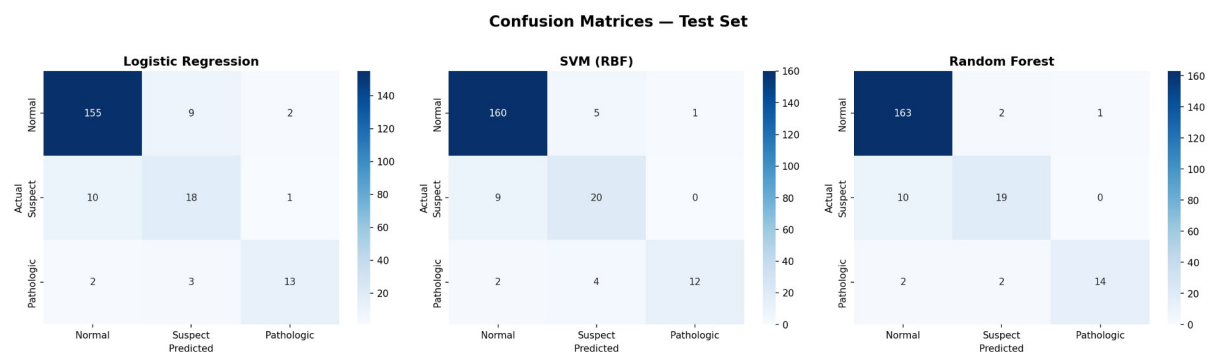
I computed metrics on the test set (213 samples). Precision, Recall, and F1-Score use macro averaging, giving equal weight to all classes regardless of frequency. ROC AUC is computed with a One-vs-Rest strategy.

Model	Accuracy	Precision	Recall	F1-Score	ROC AUC
Logistic Regression	0.873	0.780	0.759	0.769	0.967
SVM (RBF)	0.901	0.850	0.773	0.805	0.968
Random Forest	0.920	0.897	0.805	0.845	0.965
Deep Learning (DNN)	0.911	0.861	0.836	0.848	0.973

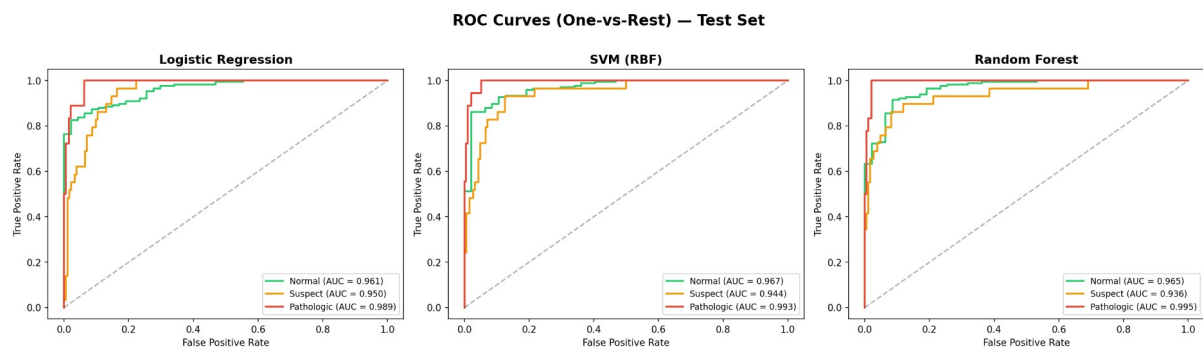
Random Forest is the best among shallow models on almost all metrics. The DNN shows a slight numerical advantage in F1-score and ROC AUC, but the margin over Random Forest is small and should be interpreted cautiously. SVM has the highest precision after RF but lower recall: in practice it is more “conservative”—when it predicts Pathologic it is usually correct, but it misses more among the minority classes.

7. Task 4 - Confusion matrices and ROC curves

The confusion matrices show a common pattern across all models: Normal cases are classified very well (over 95%), but the Suspect class is often confused with Normal. This makes clinical sense: suspect cases are by definition borderline.



The ROC curves confirm that all models have excellent discriminative ability, with AUC above 0.95 for all classes. Normal is the easiest class to identify; Pathologic has slightly lower AUC but still very high.



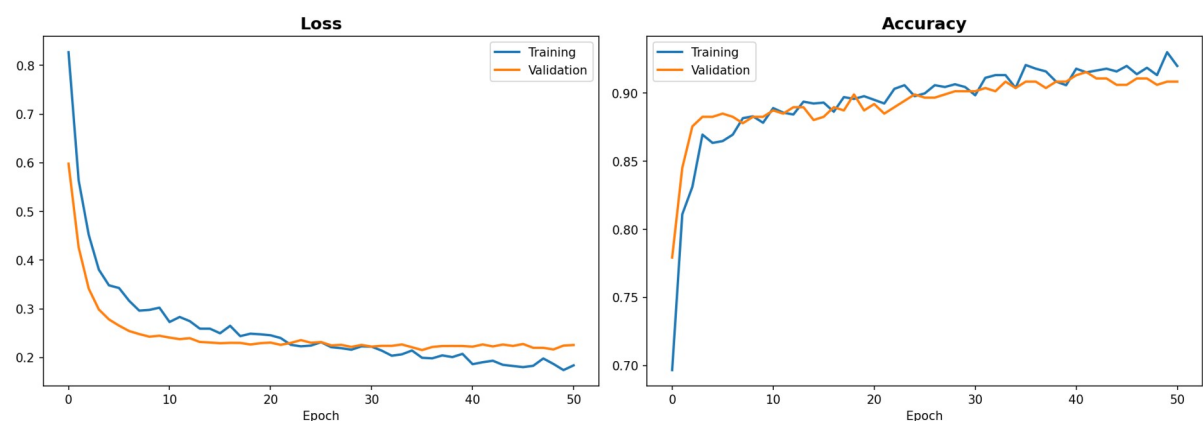
8. Task 5a - Deep Learning

I implemented a fully connected neural network with TensorFlow/Keras. The architecture I chose is:

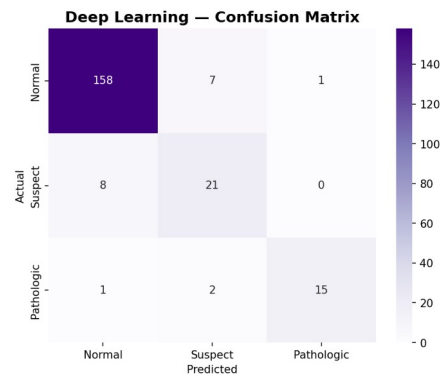
Input(14) → Dense(64, ReLU) → Dropout(0.3) → Dense(32, ReLU) → Dropout(0.2) → Dense(3, softmax)

I used Dropout layers to regularize and limit overfitting, since the dataset is not very large. The optimizer is Adam with categorical cross-entropy as the loss. I set EarlyStopping on validation loss so training stops when generalization stops improving.

PCA was kept for pipeline consistency and educational purposes; this does not imply optimality for all models, especially tree-based ensembles.



On the test set, the Deep Learning model reaches Accuracy 0.9108, macro F1-Score 0.8477, and ROC AUC 0.9729: in the test-set comparison it is slightly ahead on macro F1 and ROC AUC, but performance remains very close to Random Forest, which is still highly competitive and easier to interpret via feature importance.

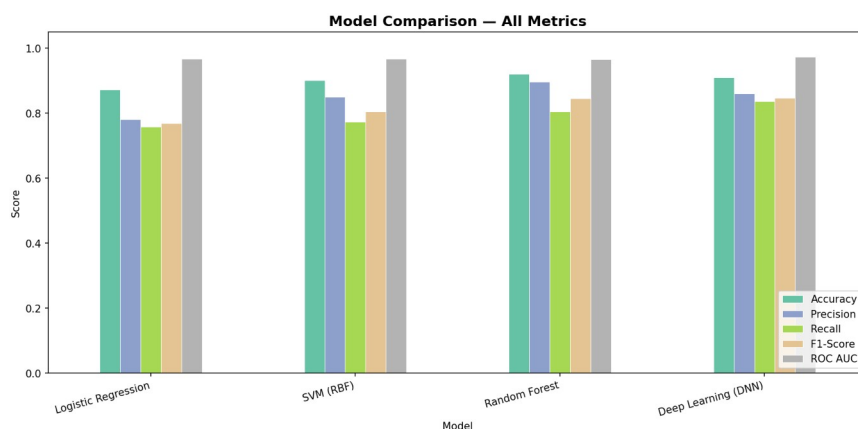


9. Model comparison

Looking at the comparison table and chart, the four models have similar but not identical performance. On the test set, Deep Learning (DNN) and Random Forest show broadly comparable results; the DNN has a slight numerical edge on macro F1 and ROC AUC.

A common issue across all models is difficulty with the Suspect class, often confused with Normal: this depends not only on the chosen algorithm, but on overlap between “normal” and “suspicious” profiles in the dataset.

This highlights an important point: the bottleneck is not the algorithm, but the data itself. Normal and Suspect have overlapping clinical profiles, and no model, however sophisticated, can separate them completely.



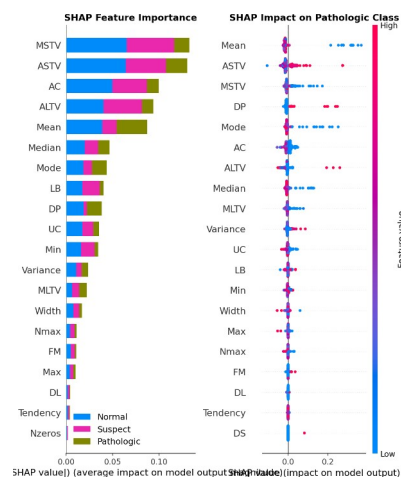
10. Task 5b - Explainability

For explainability I used two levels, both on the Random Forest trained on standardized original features (without PCA), so variables remain interpretable (e.g. ASTV, MSTV, AC) rather than abstract principal components.

First, I analyzed Random Forest feature importance (feature_importances_). In this run, the highest-weight variables are mainly MSTV, ASTV, ALTV, Mean, and Median (ordered by RF importances).

Then I applied SHAP with TreeExplainer on the same Random Forest. SHAP can emphasize variables slightly differently from mean importances, because it explains contributions more “locally” and can vary by predicted class.

Overall, short/long-term variability (e.g. MSTV/ASTV/ALTV) and signals related to heart-rate dynamics (e.g. linked to accelerations and histogram patterns) are among the most relevant factors from the analysis. This does not replace clinical judgment, but helps clarify which variables the model weights most in its decisions.



11. Conclusions

This project showed me how to build a complete machine learning pipeline, from preprocessing to interpreting results. A few points that stood out:

- PCA confirms substantial redundancy among features: from 21 original features to 14 components for 95% variance, consistent with observed correlations (e.g. Mode/Mean/Median).
- On the test set, DNN and Random Forest achieve very similar results: the DNN has a slight edge in macro F1 (0.848) and ROC AUC (0.973), while Random Forest remains the best among shallow models and easier to interpret.
- The explainability section is most useful when read at two levels: RF importances (simple) and SHAP (more detailed). This clarifies what drives predictions.
- Priority future improvements: explicit handling of class imbalance (e.g. class_weight), more robust validation (cross-validation), and systematic hyperparameter tuning.
- Limitations: the test set contains 213 samples, including 18 Pathologic; for this class, metrics can shift noticeably with only a few different predictions. Results should therefore be read as indicative for this single split, without cross-validation.

Appendix — Python code

Below is the complete Python notebook code, organized by cells.

Cell 1

```
# Step 0 - Imports
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, roc_curve, auc,
    classification_report
)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping

import shap

output_dir = 'outputs'
os.makedirs(output_dir, exist_ok=True)

print("Imports completed.")
print(f"TensorFlow version: {tf.__version__}")
print(f"Output directory: {output_dir}")
```

Cell 2

```
# Step 1 - Load dataset
# The CTG dataset from UCI (ID: 193)
df = pd.read_csv('CTG.csv')

feature_names = [
    'LB',          # FHR baseline (beats per minute)
    'AC',          # Accelerations per second
    'FM',          # Fetal movements per second
    'UC',          # Uterine contractions per second
    'DL',          # Light decelerations per second
    'DS',          # Severe decelerations per second
    'DP',          # Prolonged decelerations per second
    'ASTV',        # % time with abnormal short term variability
    'MSTV',        # Mean short term variability
    'ALTV',        # % time with abnormal long term variability
    'MLTV',        # Mean long term variability
    'Width',       # FHR histogram width
    'Min',         # FHR histogram minimum
    'Max',         # FHR histogram maximum
    'Nmax',        # Number of histogram peaks
    'Nzeros',      # Number of histogram zeros
    'Mode',        # Histogram mode
```



```

    'Mean',      # Histogram mean
    'Median',    # Histogram median
    'Variance',  # Histogram variance
    'Tendency'   # Histogram tendency
]

target_col = 'NSP'
target_labels = {1: 'Normal', 2: 'Suspect', 3: 'Pathologic'}
class_labels = ['Normal', 'Suspect', 'Pathologic']

df_clean = df[feature_names + [target_col]].dropna()
X = df_clean[feature_names].values
y = df_clean[target_col].values.astype(int)

print(f"Dataset loaded: {X.shape[0]} samples, {X.shape[1]} features")
print("Target distribution:")
for val, name in target_labels.items():
    count = (y == val).sum()
    print(f"    {name} (class {val}): {count} ({count/len(y)*100:.1f}%)")

```

Cell 3

```

# Exploratory Data Analysis
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Class distribution
class_counts = pd.Series(y).value_counts().sort_index()
colors = ['#2ecc71', '#f39c12', '#e74c3c']
bars = axes[0].bar([target_labels[i] for i in class_counts.index], class_counts.values,
color=colors)
axes[0].set_title('Fetal State Distribution', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Count')
for bar, count in zip(bars, class_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
        str(count), ha='center', fontsize=12)

# Correlation heatmap
df_features = pd.DataFrame(X, columns=feature_names)
corr = df_features.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, cmap='RdBu_r', center=0, ax=axes[1],
            square=True, linewidths=0.5, cbar_kws={'shrink': 0.8})
axes[1].set_title('Feature Correlation Matrix', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig(f'{output_dir}/01_eda.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 4

```

# Split data (70/20/10, stratified) and scale features
# 70/20/10 split with stratification
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=(2/9), random_state=42, stratify=y_train_val
)

print(f"Training set:    {X_train.shape[0]} samples ({X_train.shape[0]/len(X)*100:.0f}%)")
print(f"Validation set: {X_val.shape[0]} samples ({X_val.shape[0]/len(X)*100:.0f}%)")
print(f"Test set:         {X_test.shape[0]} samples ({X_test.shape[0]/len(X)*100:.0f}%)")

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

```

```
print("Scaling completed (fit on train only).")
```

Cell 5

```
# Task 1: PCA analysis (variance + 2D projection)
pca_full = PCA(n_components=None)
pca_full.fit(X_train_scaled)
cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
n_95 = np.argmax(cumulative_variance >= 0.95) + 1

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(range(1, len(pca_full.explained_variance_ratio_) + 1),
            pca_full.explained_variance_ratio_, alpha=0.7, label='Individual',
            color='#3498db')
axes[0].step(range(1, len(cumulative_variance) + 1), cumulative_variance,
            where='mid', label='Cumulative', color='#e74c3c', linewidth=2)
axes[0].axhline(y=0.95, color='gray', linestyle='--', alpha=0.7, label='95% threshold')
axes[0].axvline(x=n_95, color='green', linestyle='--', alpha=0.7, label=f'n={n_95}'
components')
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance Ratio')
axes[0].set_title('PCA - Explained Variance', fontsize=14, fontweight='bold')
axes[0].legend(fontsize=9)

pca_2d = PCA(n_components=2)
X_train_2d = pca_2d.fit_transform(X_train_scaled)
colors_map = {1: '#2ecc71', 2: '#f39c12', 3: '#e74c3c'}
for cls, label in target_labels.items():
    m = y_train == cls
    axes[1].scatter(X_train_2d[m, 0], X_train_2d[m, 1], c=colors_map[cls],
                    label=label, alpha=0.6, s=20, edgecolors='w', linewidth=0.3)
axes[1].set_xlabel(f'PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f}%)')
axes[1].set_ylabel(f'PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f}%)')
axes[1].set_title('PCA - 2D Projection (Training Set)', fontsize=14, fontweight='bold')
axes[1].legend()

plt.tight_layout()
plt.savefig(f'{output_dir}/02_pca.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"\n95% variance with {n_95} components (out of {X_train_scaled.shape[1]})")
print(f"First 2 PCs explain {cumulative_variance[1]*100:.1f}% of variance")
```

Cell 6

```
# Apply PCA with optimal components
pca_final = PCA(n_components=n_95)
X_train_pca = pca_final.fit_transform(X_train_scaled)
X_val_pca = pca_final.transform(X_val_scaled)
X_test_pca = pca_final.transform(X_test_scaled)
print(f"Dimensionality reduced: {X_train_scaled.shape[1]} → {X_train_pca.shape[1]} features")
```

Cell 7

```
# Task 2: define and train shallow classifiers on PCA features
# We train Logistic Regression, SVM, and Random Forest on the same transformed data.
classifiers = {
    'Logistic Regression': LogisticRegression(max_iter=1000, solver='lbfgs',
random_state=42),
    'SVM (RBF)': SVC(kernel='rbf', probability=True, random_state=42, gamma='auto'),
    'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
}

classes = sorted(np.unique(y))
y_test_bin = label_binarize(y_test, classes=classes)
```

```

results = {}
metrics_data = []

for name, model in classifiers.items():
    model.fit(X_train_pca, y_train)
    y_pred = model.predict(X_test_pca)
    y_prob = model.predict_proba(X_test_pca)
    results[name] = {'model': model, 'y_pred': y_pred, 'y_prob': y_prob}
    print(f"Model trained: {name}")

```

Cell 8

```

# Task 3: compute test metrics for each shallow classifier
for name, res in results.items():
    y_pred = res['y_pred']
    y_prob = res['y_prob']

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='macro', zero_division=0)
    rec = recall_score(y_test, y_pred, average='macro', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)
    roc = roc_auc_score(y_test_bin, y_prob, multi_class='ovr', average='macro')

    metrics_data.append({
        'Model': name,
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'F1-Score': f1,
        'ROC AUC': roc
    })

    print(f"\n--- {name} ---")
    print(f"Accuracy: {acc:.4f} | Precision: {prec:.4f} | Recall: {rec:.4f} | F1: {f1:.4f} | ROC AUC: {roc:.4f}")
    print("Classification report:")
    print(classification_report(y_test, y_pred, target_names=class_labels, zero_division=0))

metrics_df = pd.DataFrame(metrics_data).set_index('Model')
print("\n" + "=" * 60)
print("Summary metrics (test set)")
print(metrics_df.round(4).to_string())

```

Cell 9

```

# Task 4a: confusion matrices on test set
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for idx, (name, res) in enumerate(results.items()):
    cm = confusion_matrix(y_test, res['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
                xticklabels=class_labels, yticklabels=class_labels)
    axes[idx].set_title(name, fontsize=13, fontweight='bold')
    axes[idx].set_xlabel('Predicted'); axes[idx].set_ylabel('Actual')
plt.suptitle('Confusion Matrices - Test Set', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(f'{output_dir}/03_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 10

```

# Task 4b: ROC curves (one-vs-rest)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
colors_roc = ['#2ecc71', '#f39c12', '#e74c3c']
for idx, (name, res) in enumerate(results.items()):
    for i, cls in enumerate(classes):

```

```

        fpr, tpr, _ = roc_curve(y_test_bin[:, i], res['y_prob'][:, i])
        roc_val = auc(fpr, tpr)
        axes[idx].plot(fpr, tpr, color=colors_roc[i], linewidth=2,
                        label=f'{class_labels[i]} (AUC = {roc_val:.3f})')
        axes[idx].plot([0, 1], [0, 1], 'k--', alpha=0.3)
        axes[idx].set_xlabel('False Positive Rate'); axes[idx].set_ylabel('True Positive Rate')
        axes[idx].set_title(name, fontsize=13, fontweight='bold')
        axes[idx].legend(loc='lower right', fontsize=9)
plt.suptitle('ROC Curves (One-vs-Rest) – Test Set', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(f'{output_dir}/04_roc_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 11

```

# Task 5a: deep learning model (DNN)
# We keep the same PCA inputs and train/val/test split used above.
y_train_dl = to_categorical(y_train - 1, num_classes=3)
y_val_dl = to_categorical(y_val - 1, num_classes=3)
y_test_dl = to_categorical(y_test - 1, num_classes=3)

model_dl = Sequential([
    Input(shape=(X_train_pca.shape[1],)),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(3, activation='softmax')
])
model_dl.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

print("Model Architecture:")
model_dl.summary()

early_stop = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True)
history = model_dl.fit(
    X_train_pca, y_train_dl,
    validation_data=(X_val_pca, y_val_dl),
    epochs=150, batch_size=32, callbacks=[early_stop], verbose=1
)
print(f"\nTraining stopped at epoch {len(history.history['loss'])}")

```

Cell 12

```

# Training curves
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(history.history['loss'], label='Training', lw=2)
axes[0].plot(history.history['val_loss'], label='Validation', lw=2)
axes[0].set_title('Loss', fontsize=14, fontweight='bold'); axes[0].set_xlabel('Epoch');
axes[0].legend()
axes[1].plot(history.history['accuracy'], label='Training', lw=2)
axes[1].plot(history.history['val_accuracy'], label='Validation', lw=2)
axes[1].set_title('Accuracy', fontsize=14, fontweight='bold'); axes[1].set_xlabel('Epoch');
axes[1].legend()
plt.tight_layout()
plt.savefig(f'{output_dir}/05_dl_training_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 13

```

# DL Evaluation
y_prob_dl = model_dl.predict(X_test_pca)
y_pred_dl = np.argmax(y_prob_dl, axis=1) + 1

acc_dl = accuracy_score(y_test, y_pred_dl)
prec_dl = precision_score(y_test, y_pred_dl, average='macro', zero_division=0)

```

```

rec_dl = recall_score(y_test, y_pred_dl, average='macro', zero_division=0)
f1_dl = f1_score(y_test, y_pred_dl, average='macro', zero_division=0)
roc_dl = roc_auc_score(y_test_bin, y_prob_dl, multi_class='ovr', average='macro')

print("Deep Learning (DNN) – Test Set Results")
print(f" Accuracy: {acc_dl:.4f} Precision: {prec_dl:.4f}")
print(f" Recall: {rec_dl:.4f} F1-Score: {f1_dl:.4f}")
print(f" ROC AUC: {roc_dl:.4f}")

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_dl), annot=True, fmt='d', cmap='Purples',
              ax=ax, xticklabels=class_labels, yticklabels=class_labels)
ax.set_title('Deep Learning – Confusion Matrix', fontsize=14, fontweight='bold')
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
plt.tight_layout()
plt.savefig(f'{output_dir}/06_dl_confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 14

```

# Compare shallow models vs DNN
dl_row = {'Model': 'Deep Learning (DNN)', 'Accuracy': acc_dl, 'Precision': prec_dl,
          'Recall': rec_dl, 'F1-Score': f1_dl, 'ROC AUC': roc_dl}
all_metrics = metrics_data + [dl_row]
comparison_df = pd.DataFrame(all_metrics).set_index('Model')

print("="*70)
print("FINAL COMPARISON – ALL MODELS")
print("="*70)
print(comparison_df.round(4).to_string())
print(f"\nBest by F1-Score: {comparison_df['F1-Score'].idxmax()} ({comparison_df['F1-Score'].max():.4f})")

fig, ax = plt.subplots(figsize=(12, 6))
comparison_df.plot(kind='bar', ax=ax, colormap='Set2', edgecolor='white', linewidth=0.5)
ax.set_title('Model Comparison – All Metrics', fontsize=14, fontweight='bold')
ax.set_ylabel('Score'); ax.set_ylim(0, 1.05)
ax.set_xticklabels(ax.get_xticklabels(), rotation=15, ha='right')
ax.legend(loc='lower right')
plt.tight_layout()
plt.savefig(f'{output_dir}/07_model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 15

```

# Task 5b: explainability on original features
rf_original = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
rf_original.fit(X_train_scaled, y_train)

# 1) Random Forest feature importances
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': rf_original.feature_importances_
}).sort_values('Importance', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df.head(10), x='Importance', y='Feature', color='#4c72b0')
plt.title('Random Forest Feature Importances (Top 10)', fontsize=13, fontweight='bold')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.savefig(f'{output_dir}/08_rf_feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()

print('Top 5 features by Random Forest importance:')
print(importance_df.head(5).to_string(index=False))

```

```

# 2) SHAP analysis
X_test_df = pd.DataFrame(X_test_scaled, columns=feature_names)
explainer = shap.TreeExplainer(rf_original)
shap_values = explainer.shap_values(X_test_scaled)
shap_values = np.array(shap_values)

# Handle SHAP output format: (samples, features, classes)
if shap_values.shape[0] == X_test_scaled.shape[0]:
    sv_list = [shap_values[:, :, i] for i in range(shap_values.shape[2])]
    sv_pathologic = shap_values[:, :, 2]
else:
    sv_list = [shap_values[i] for i in range(3)]
    sv_pathologic = shap_values[2]

print(f"SHAP values computed. Shape: {shap_values.shape}")

```

Cell 16

```

# SHAP summary plots
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

plt.sca(axes[0])
shap.summary_plot(sv_list, X_test_df, plot_type='bar', class_names=class_labels, show=False)
axes[0].set_title('SHAP Feature Importance', fontsize=13, fontweight='bold')

plt.sca(axes[1])
shap.summary_plot(sv_pathologic, X_test_df, show=False)
axes[1].set_title('SHAP Impact on Pathologic Class', fontsize=13, fontweight='bold')

plt.tight_layout()
plt.savefig(f'{output_dir}/09_shap_original_features.png', dpi=150, bbox_inches='tight')
plt.show()

print('SHAP analysis completed.')
print('Compare these results with RF feature_importances_ for consistency.')

```